Begin

Nicholas Alan Gilbert Emperical Exercise #2 10-26-26

Table of Contents

Set Up

Data Handling

Task 1 - Compute Log Returns

Task 2 - Annual Returns and Volatility

Task 3 - Data Visualizations

Set Up

We will import necessary libraries and custom helper-functions written in another file.

In [63]:
```
%load_ext autoreload
%autoreload 2

import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as st
import numpy as np
import ipywidgets

#Custom Helper Functions
import helper.tools as tools
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

In [125…
```
stocks = tools.StockSearch()
```

```
VBox(children=(HBox(children=(Combobox(value='', description='Ticker:',
options=('MMM', 'AOS', 'ABT', 'ABBV', …
```

In [232…
```
stocks.start
```

Out[232… datetime.date(2000, 1, 1)

In [233…
```
stocks.end
```

Out[233…
```
datetime.date(2026, 10, 26)
```

In [234…
```
stocks.tickers
```

Out[234…
```
['AAPL', 'TRMD', 'LMT', 'ADBE', 'MSFT', 'NVDA', 'AMD', 'XLF']
```

In [235…
```
df = stocks.data.copy()
```

```
[**********100%**********]  8 of 8 completed
```

In [236…
```
df
```

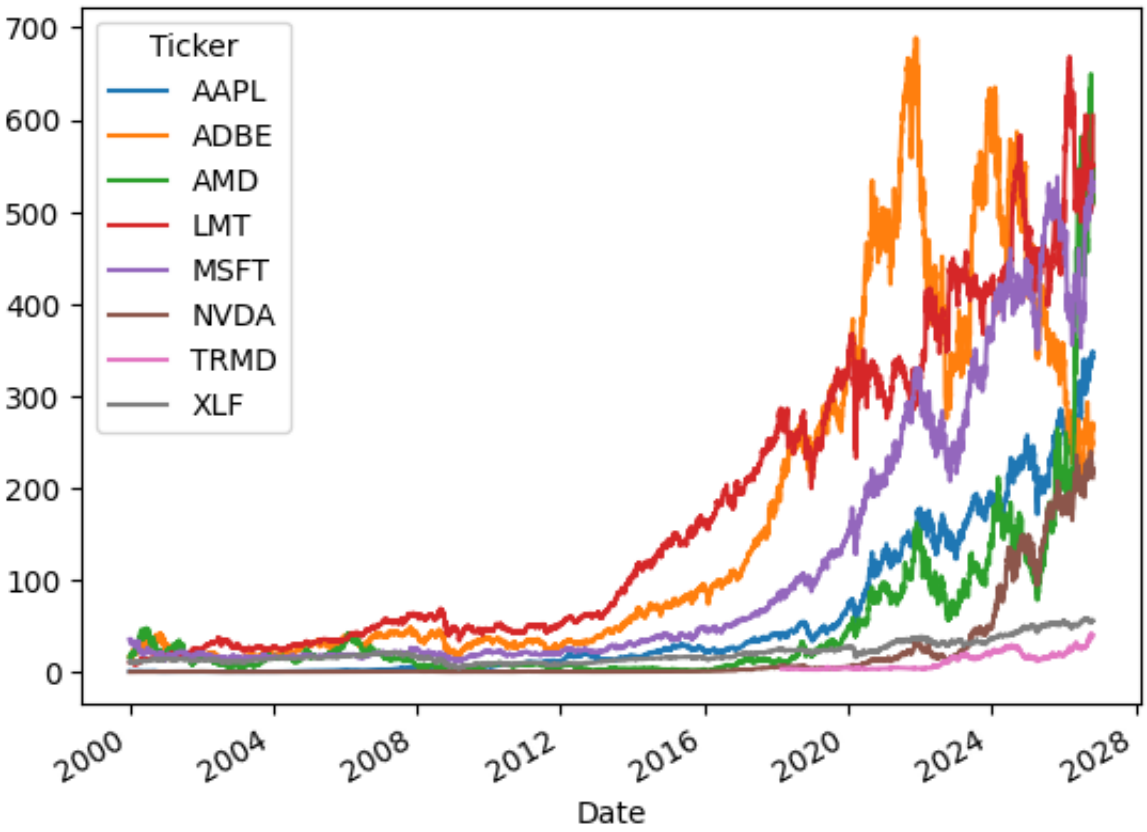
Out[236…

Price						
Ticker	AAPL	ADBE	AMD	LMT	MSFT	NV
Date						
2000-01-03	0.837002	16.274670	15.500000	10.281113	35.377014	0.089
2000-01-04	0.766435	14.909398	14.625000	10.630160	34.181980	0.086
2000-01-05	0.777650	15.204174	15.000000	10.693622	34.542385	0.083
2000-01-06	0.710354	15.328289	16.000000	10.852289	33.385277	0.078
2000-01-07	0.744002	16.072983	16.250000	10.884013	33.821556	0.079
…	…	…	…	…	…	
2026-10-19	342.940002	247.089996	538.049988	548.229980	533.520020	217.089
2026-10-20	339.500000	256.730011	534.739990	548.710022	531.869995	216.470
2026-10-21	343.989990	255.059998	552.719971	551.510010	532.950012	221.970
2026-10-22	348.220001	270.549988	507.929993	589.909973	521.080017	212.210
2026-10-23	347.730011	263.820007	516.789978	604.179993	524.510010	210.960

6743 rows × 16 columns

In [237…
```
df['Close'].plot()
```

Out[237… <Axes: xlabel='Date'>



Data Handling

Back to Top

Log Returns

Log returns are computed under the assumption of *continous time*. It comes from the inverse of the exponential function e^t

$\ln(e^t) = t$

Read: the natural-log of e *to the something* is that something.

Context

Where We Definied Returns Before;

$ r_t = $ return at time period t

$ p_t = $ asset price at time t

$ t = $ time-period eg. daily, weekly, monthly

Asset Return:

$$ r_t = \frac{p_t}{p_{t-1}} - 1$$

Then a series of successive returns of n periods compounded is represented geometrically:

$$r_{\text{cumulative}} := (\prod_{i=1}^{n} 1 + r_i) - 1 = [(1+r_1) \cdot (1+r_2) \cdot (1+r_3) \dots (1+r_n)] - 1$$

This is a more expensive operation, adding 1s to each entry then performing n multiplications.

Natural Log to the Rescue

Some nice arithmetic properties of using ln() returns is that compounding is additive.

$$\ln\left(\frac{p_t}{p_{t-1}} \right) = \ln(p_t) - \ln(p_{t-1})$$

$$\ln\left({p_t}\cdot{p_{t-1}} \right) = \ln(p_t) + \ln(p_{t-1})$$

We Define Log-Returns:

$$r_t := \ln(p_t) - \ln(p_{t-1})$$

$$r_t := \ln(p_{\text{today} }) - \ln(p_{\text{yesterday} })$$

Calculation Method

We can apply NumPy's `ln()` function to an entire array of asset prices, then use pandas' `.shift()` method to subract a whole shifted array from the other.

Will look something like this:

```
log_returns = np.log(data['Close']) -
np.log(data['Close']).shift(1)
```

Missing Data

At this point we will 'produce' a NaN value in our return calculation from our first date, we don't have a previous data point to compute a return from that point. So

we'll just drop it. Other stock's will exhbit NaN's, say they weren't listed on an exchange at that point, so no data occurs for them.

Interpretation

Working with log-returns is nice mathematically. But it will differ from the true return an investor will see if they have their money invested. For example, between 2018-02-26 and 2026-08-28.

Example

AAPL's Average Daily Log-Return: 0.096%

AAPL's Average Daily Real Return: 0.123%

In [ ]:

Task 1

Compute Log-Returns

In [238…

```
log_returns = np.log(df['Close']) - np.log(df['Close']).shift(1)
log_returns
```

Out [238…

Ticker	AAPL	ADBE	AMD	LMT	MSFT	NVDA	
Date							
2000-01-03	NaN	NaN	NaN	NaN	NaN	NaN	
2000-01-04	-0.088077	-0.087618	-0.058108	0.033387	-0.034364	-0.027063	
2000-01-05	0.014527	0.019578	0.025318	0.005952	0.010489	-0.033476	
2000-01-06	-0.090514	0.008130	0.064539	0.014729	-0.034072	-0.067479	
2000-01-07	0.046281	0.047440	0.015504	0.002919	0.012983	0.016562	
...	...	...	...	...	...	...	
2026-10-19	-0.001515	0.007678	0.017436	-0.007016	0.008414	0.013729	-0.0
2026-10-20	-0.010082	0.038272	-0.006171	0.000875	-0.003098	-0.002860	0.0
2026-10-21	0.013139	-0.006526	0.033071	0.005090	0.002029	0.025090	0.0
2026-10-22	0.012222	0.058958	-0.084508	0.067310	-0.022524	-0.044966	-0.0
2026-10-23	-0.001408	-0.025190	0.017293	0.023902	0.006561	-0.005908	0.0

6743 rows × 8 columns

In [239…

```
log_returns.isna().sum(), log_returns[log_returns.index > '2018'].isna
```

Out[239…
```
(Ticker
 AAPL       1
 ADBE       1
 AMD        1
 LMT        1
 MSFT       1
 NVDA       1
 TRMD    4565
 XLF        1
 dtype: int64,
 Ticker
 AAPL     0
 ADBE     0
 AMD      0
 LMT      0
 MSFT     0
 NVDA     0
 TRMD    37
 XLF      0
 dtype: int64)
```

Note on NaN Handling

So far we can just ignore the NaNs. They aren't 'in the way' for calculating log-returns at this point. The 'sanity check' above tells us there isn't large holes in the data. I know that ticker TRMD wasn't listed until 2018 compared to our other tickers. So we are missing data for historical reasons. Plus we will have NaN from calculating returns.

If we know some caclulations will choose to 'ignore NaNs' or 'include NaNs', this is what looking up the code documentation is for.

In [240…
```
sum_stats = tools.summary_statistics_table(log_returns)

sum_stats
```

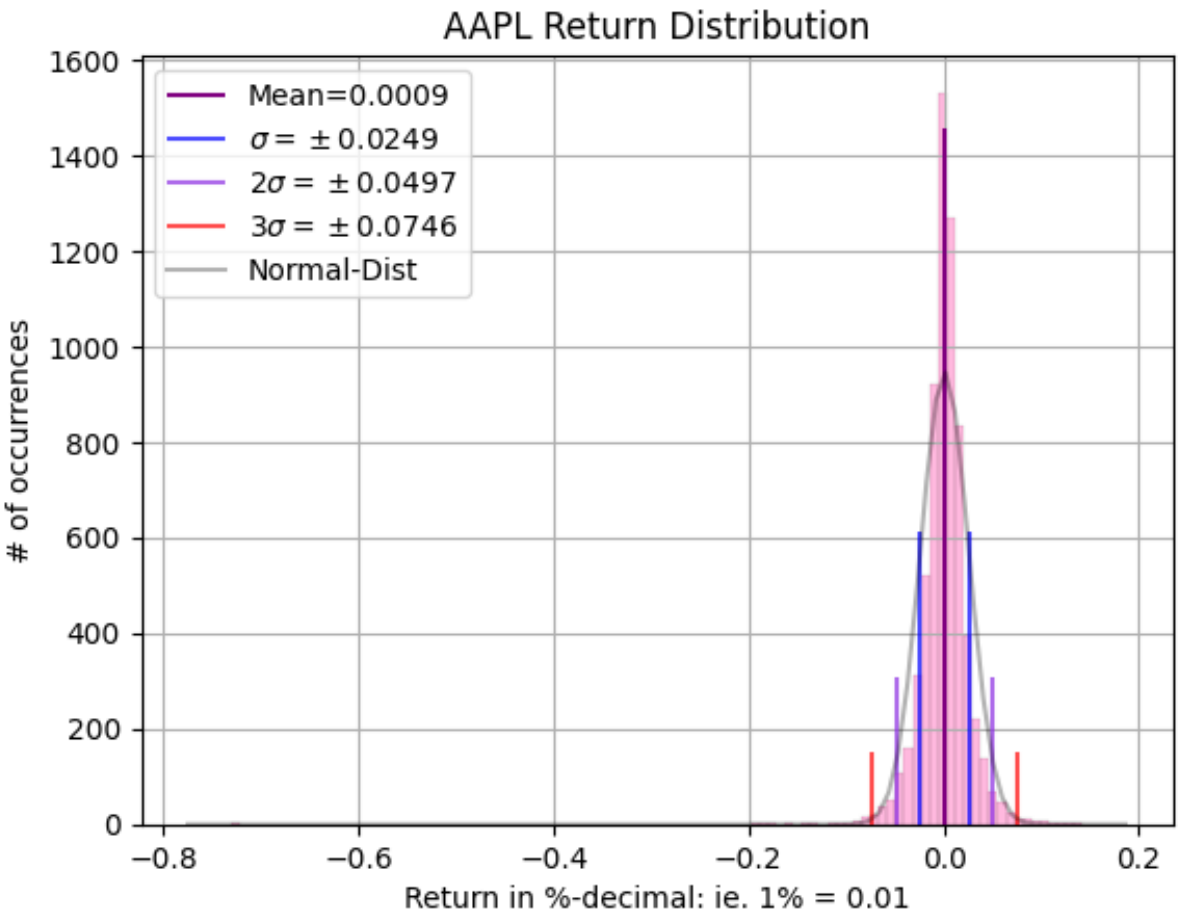
Out[240…

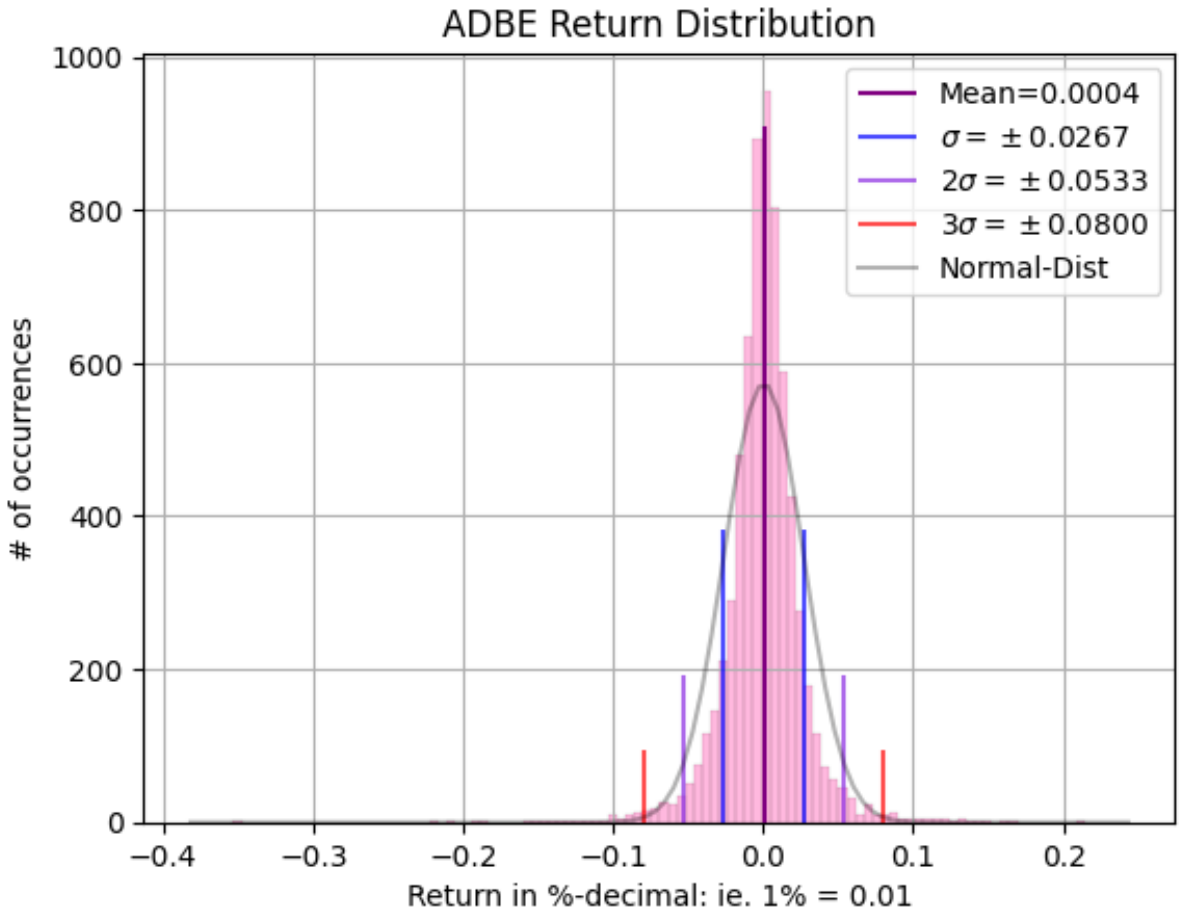
	AAPL	ADBE	AMD	LMT	MSFT	NVDA
Mean	0.000894	0.000413	0.000520	0.000604	0.000400	0.001152
Median	0.000951	0.000747	0.000000	0.000575	0.000381	0.001378
Standard-Deviation	0.024859	0.026656	0.038870	0.015886	0.018993	0.036740
Skewness	-3.917443	-0.565404	-0.085206	-0.120945	-0.090053	-0.208016
Kurtosis	115.268262	12.353597	8.187625	9.033611	9.478060	13.031658
Min	-0.731247	-0.353223	-0.391595	-0.136525	-0.169576	-0.434384
Max	0.142617	0.214885	0.420617	0.137081	0.178692	0.353574

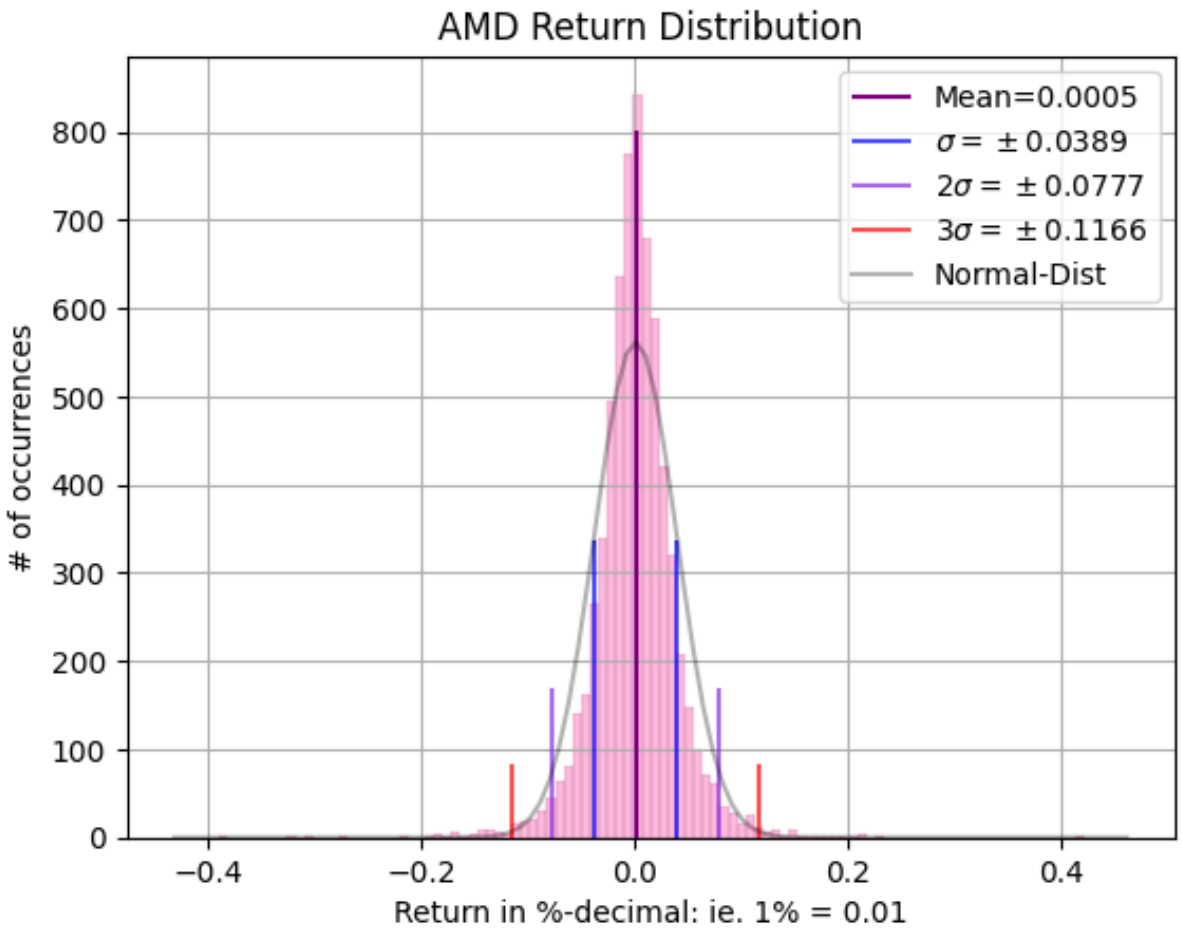
In [157…
```
np.exp(sum_stats.loc['Mean']) - 1
```

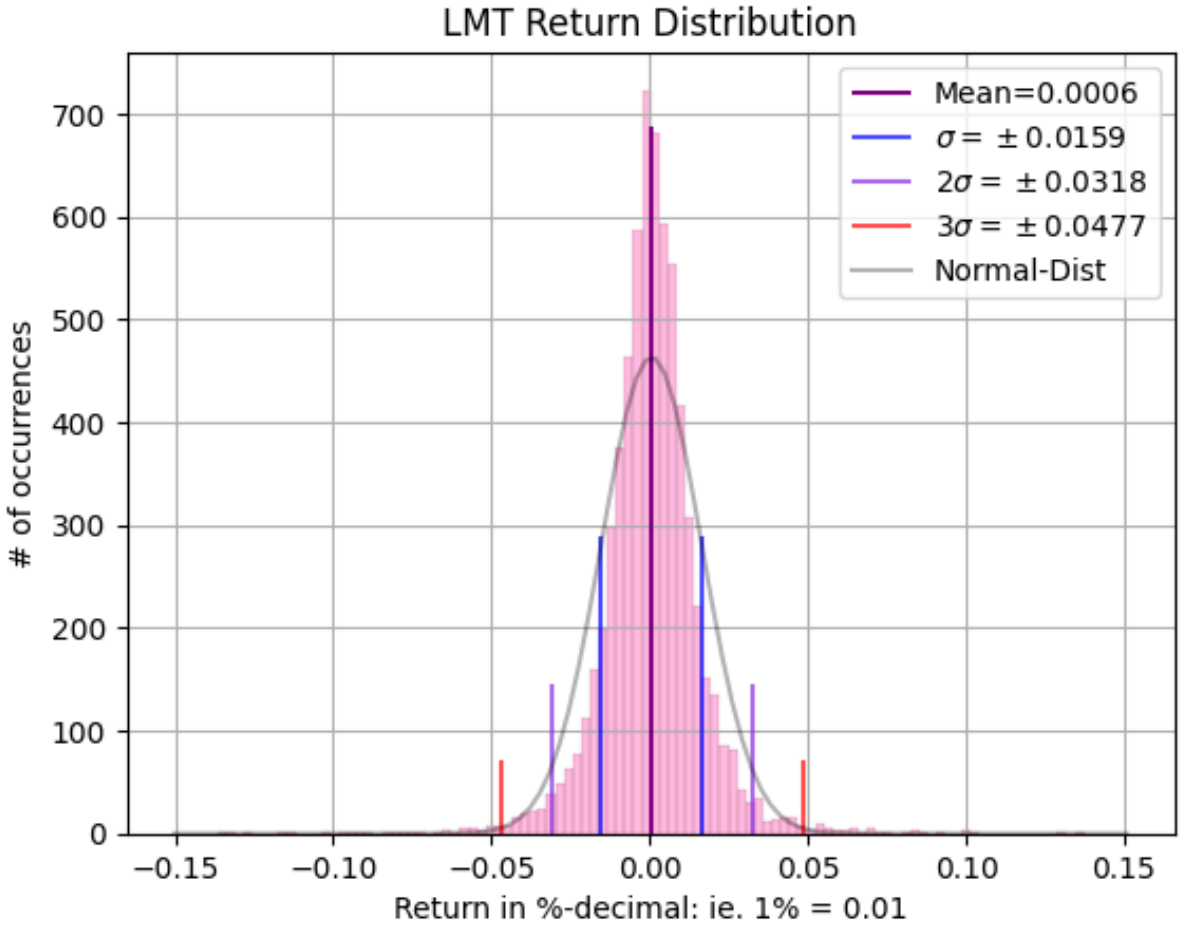
Out[157…
```
AAPL    0.000895
LMT     0.000604
TRMD    0.001143
Name: Mean, dtype: float64
```

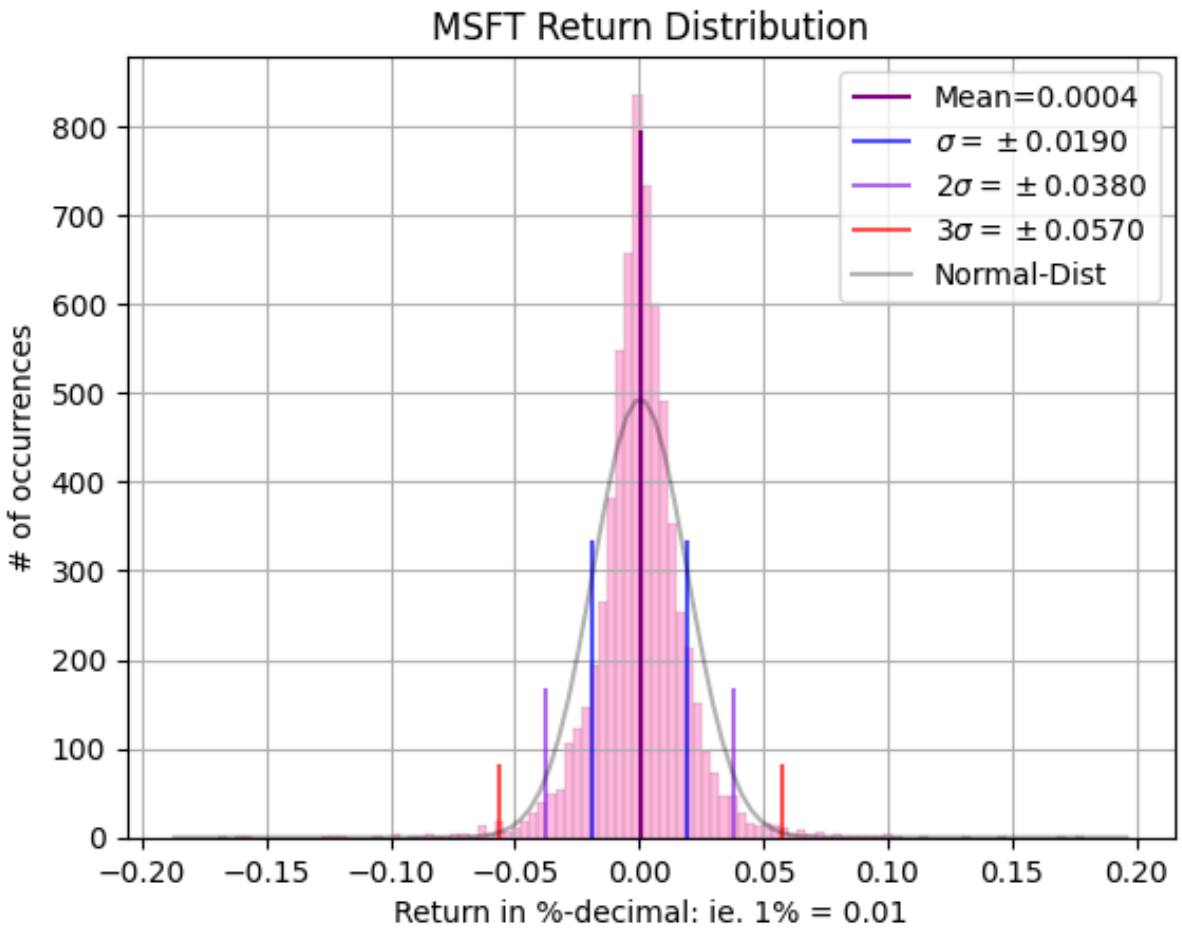
In [241…
```
tools.return_histograms(log_returns)
```

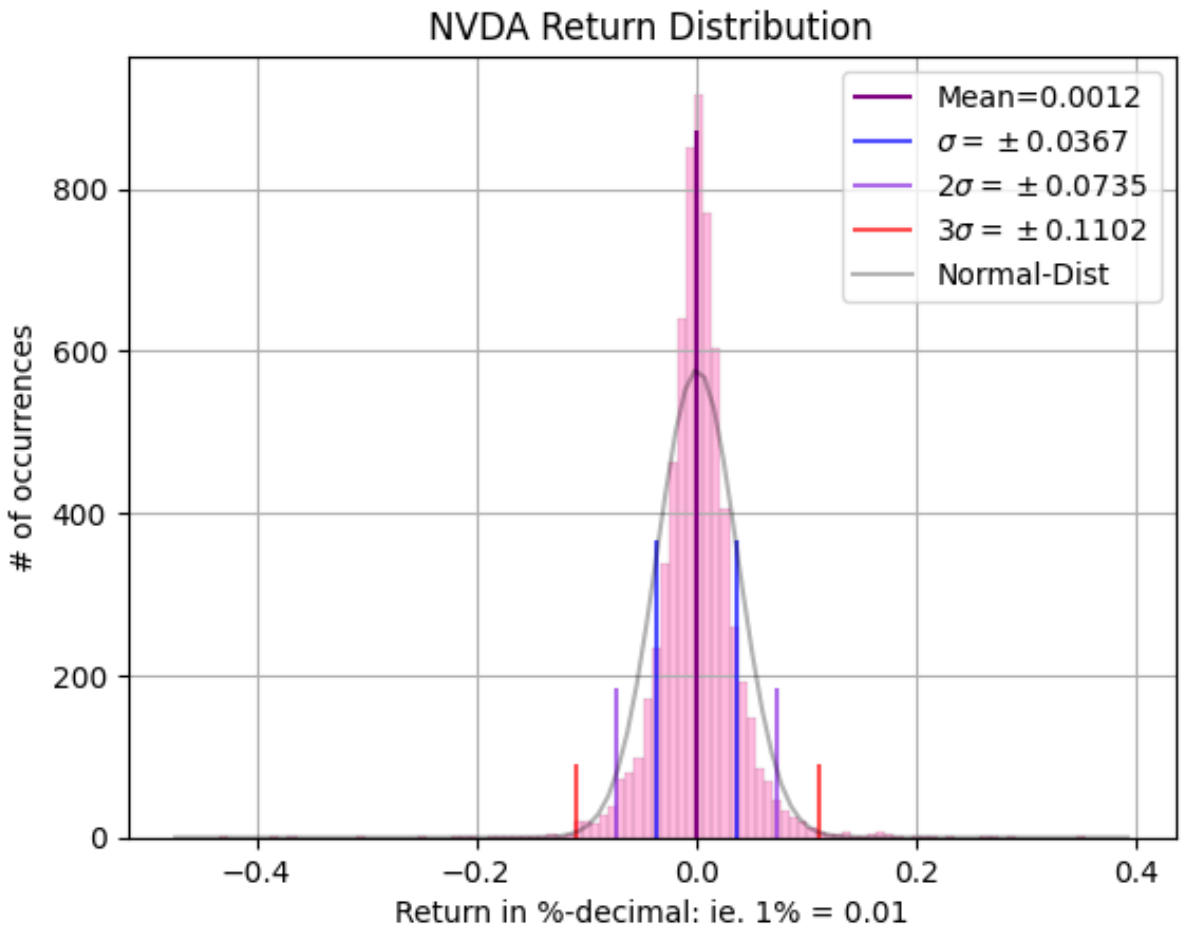


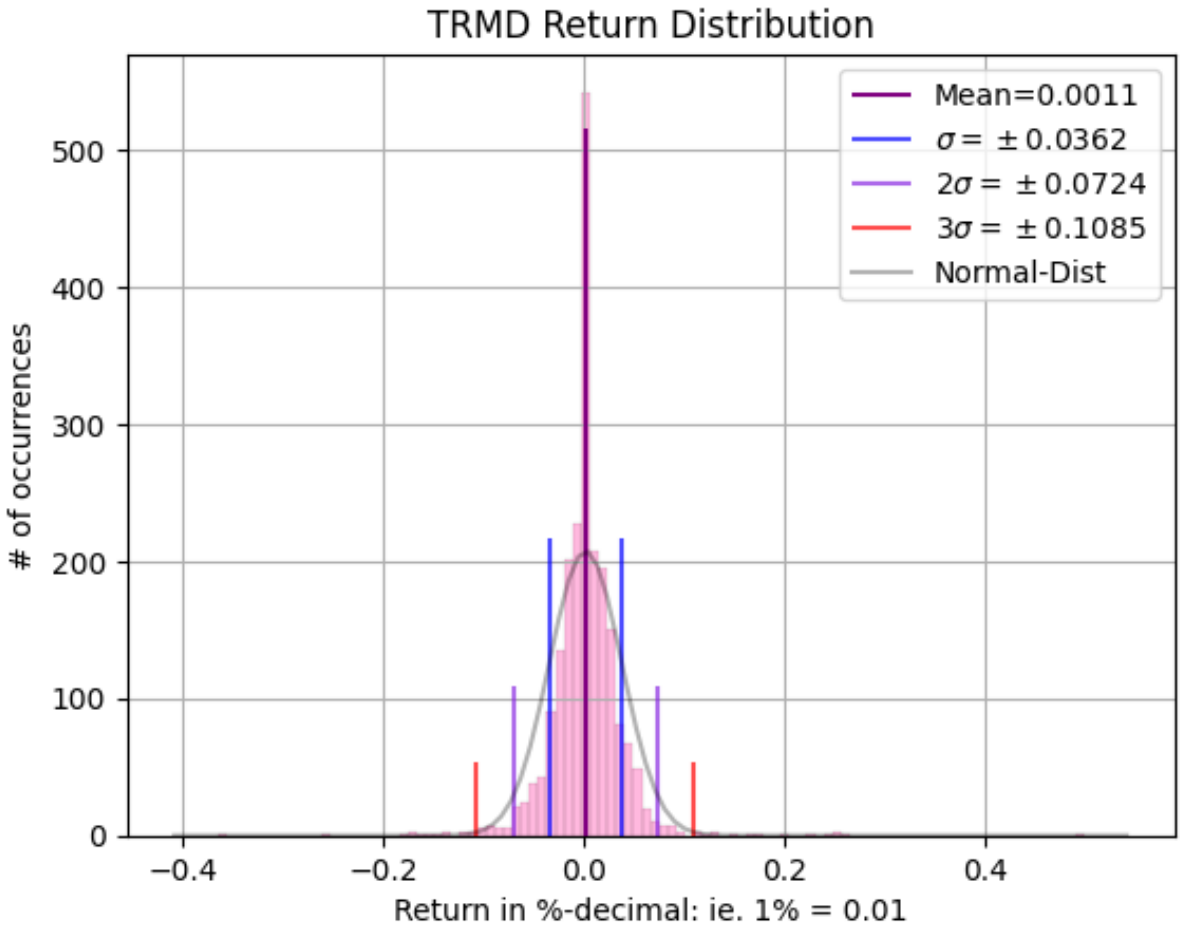

ADBE Return Distribution
Mean=0.0004
$\sigma = \pm 0.0267$
$2\sigma = \pm 0.0533$
$3\sigma = \pm 0.0800$
Normal-Dist
of occurrences
0
200
400
600
800
1000
−0.4
−0.3
−0.2
−0.1
0.0
0.1
0.2
Return in %-decimal: ie. 1% = 0.01

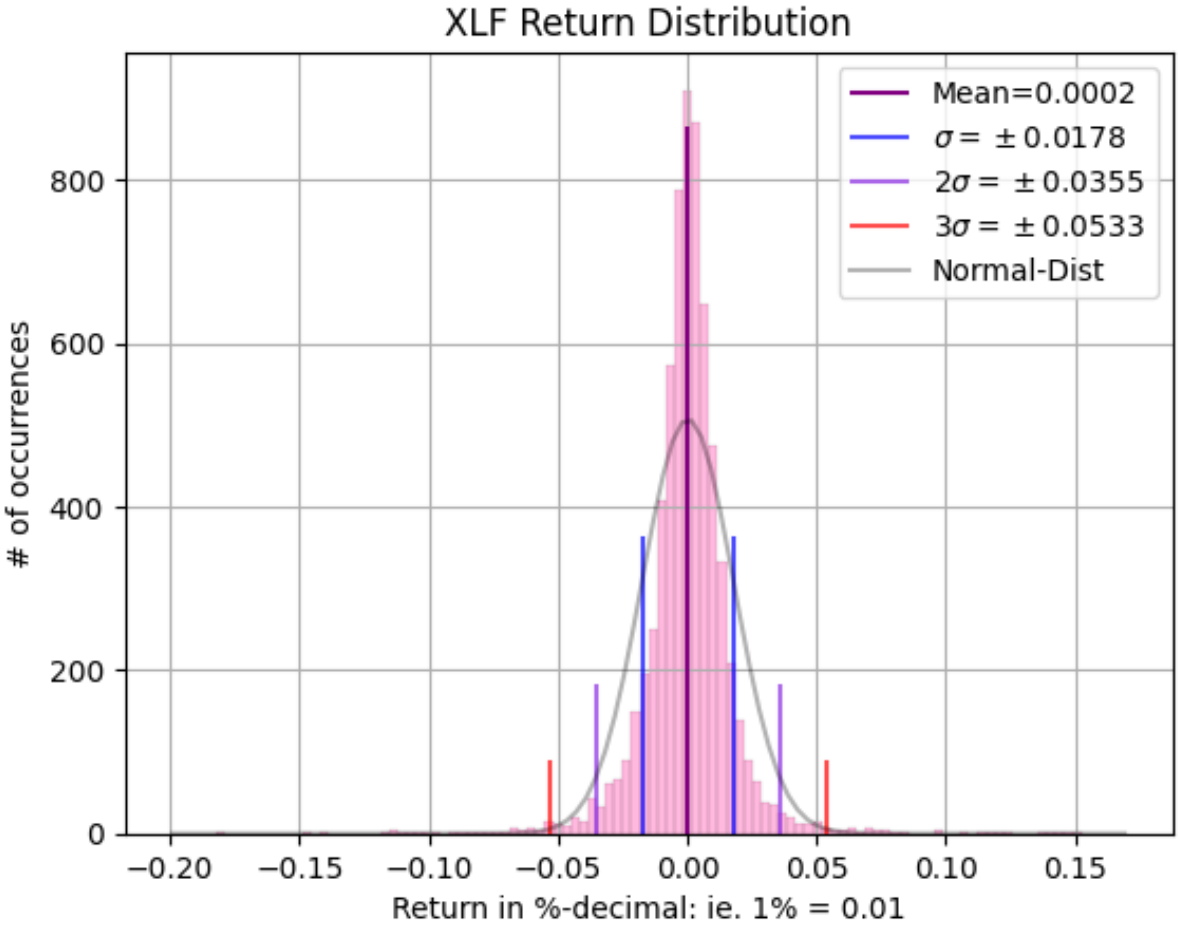

AMD Return Distribution
Mean=0.0005
$\sigma = \pm 0.0389$
$2\sigma = \pm 0.0777$
$3\sigma = \pm 0.1166$
Normal-Dist
of occurrences
0
100
200
300
400
500
600
700
800
−0.4
−0.2
0.0
0.2
0.4
Return in %-decimal: ie. 1% = 0.01


LMT Return Distribution
Mean=0.0006
σ = ±0.0159
2σ = ±0.0318
3σ = ±0.0477
Normal-Dist
of occurrences
0
100
200
300
400
500
600
700
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01


MSFT Return Distribution
Mean=0.0004
$\sigma = \pm 0.0190$
$2\sigma = \pm 0.0380$
$3\sigma = \pm 0.0570$
Normal-Dist
of occurrences
0
100
200
300
400
500
600
700
800
−0.20
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
0.20
Return in %-decimal: ie. 1% = 0.01


NVDA Return Distribution
Mean=0.0012
σ = ±0.0367
2σ = ±0.0735
3σ = ±0.1102
Normal-Dist
of occurrences
800
600
400
200
0
−0.4
−0.2
0.0
0.2
0.4
Return in %-decimal: ie. 1% = 0.01


TRMD Return Distribution
Mean=0.0011
$\sigma = \pm 0.0362$
$2\sigma = \pm 0.0724$
$3\sigma = \pm 0.1085$
Normal-Dist
of occurrences
0
100
200
300
400
500
−0.4
−0.2
0.0
0.2
0.4
Return in %-decimal: ie. 1% = 0.01



Task 2

Annual Volatility

Since log-returns are additive for compounding. We just need to group by year and sum for annualized returns across our time period.

Volatility I will elect to take rolling 90-day windows daily, then compute the average annually, then scale up to annualized by multiplying by $\sqrt{252}.$ Because volatility scales by $\sqrt{\text{time}}$.

In [242…

```
df_vol = log_returns.groupby(log_returns.index.year).std() * np.sqrt(2
df_vol
```

Out [242…

Ticker	AAPL	ADBE	AMD	LMT	MSFT	NVDA	TRMI
Date							
2000	1.013291	0.887646	0.854163	0.444989	0.568322	1.186189	NaN
2001	0.623043	0.822725	0.900010	0.325891	0.438939	0.902450	NaN
2002	0.489064	0.692421	0.952393	0.367730	0.427174	0.981401	NaN
2003	0.368728	0.408799	0.539770	0.249781	0.291202	0.667258	NaN
2004	0.396565	0.298697	0.461990	0.174354	0.170639	0.634566	NaN
2005	0.387664	0.328790	0.476979	0.152657	0.143365	0.365248	NaN
2006	0.381431	0.325087	0.516663	0.161712	0.210594	0.475017	NaN
2007	0.375966	0.243814	0.375896	0.226210	0.227153	0.463281	NaN
2008	0.587321	0.557204	0.854952	0.392372	0.480072	0.860922	NaN
2009	0.337117	0.461454	0.747912	0.337798	0.372304	0.590599	NaN
2010	0.266856	0.387890	0.452773	0.184714	0.219773	0.433601	NaN
2011	0.262498	0.337804	0.554851	0.217149	0.234736	0.572293	NaN
2012	0.293047	0.233939	0.543149	0.146997	0.207030	0.318067	NaN
2013	0.289052	0.246303	0.510984	0.163837	0.251409	0.215777	NaN
2014	0.216682	0.255751	0.415048	0.171165	0.189913	0.251187	NaN
2015	0.267491	0.235262	0.553386	0.167759	0.280299	0.338892	NaN
2016	0.233725	0.247617	0.787210	0.168902	0.227149	0.444309	NaN
2017	0.175283	0.214611	0.595090	0.111503	0.147292	0.399248	NaN
2018	0.287745	0.354572	0.631398	0.240515	0.282352	0.503993	0.55152
2019	0.262970	0.238078	0.526421	0.164202	0.198203	0.407911	0.68297
2020	0.466763	0.478132	0.608085	0.413867	0.439912	0.581173	0.95360
2021	0.250920	0.296812	0.423363	0.202473	0.210195	0.446810	0.45849
2022	0.356347	0.457323	0.613248	0.263290	0.353215	0.631647	0.60267
2023	0.202813	0.317865	0.464017	0.179892	0.249601	0.467705	0.45407
2024	0.225953	0.372241	0.481054	0.170214	0.200676	0.520646	0.33682
2025	0.322133	0.311297	0.588068	0.268496	0.240342	0.498238	0.42562
2026	0.259569	0.444757	0.705254	0.310494	0.348479	0.375230	0.41979

In [243…

```
df_yearly = df['Close'].groupby(df.index.year)
```

In [244…
```
yearly_returns = {}

for year in df_yearly:
    yr = year[0]
    df1 = year[1]


    lg_rtn = np.log(df1) - np.log(df1).shift(1)

    yearly_returns.update({yr: lg_rtn.sum()})

yearly_return_summary = pd.DataFrame(yearly_returns).T
yearly_return_summary = np.exp(yearly_return_summary) - 1
```

Annual Return Summary

Below is a table of annual returns.

Notable Years:

- 2000 was the Dot-Com era crash that led to large negative returns in technology companies like Apple.
- 2008 was the Great Financial Crisis, so we see stocks with a large negative return that year
- 2020, although Intra-Year 2020 saw a massive crash in Q1, the stock market recovered and rallied as a whole throughout the year as liquidity was pumped in the system and new market participants took discretionary money on stock trading, (ie. Robinhood stock-pickers).
- 2022 we can see the 'Mini Bear Market' of interest rate hiking following COVID

Calculation:

To take log-returns back into percentage returns, we need to exponentiate them.

$$r_{\text{Compounded}} = e^{r_{\text{log}} } - 1$$

In [245…
```
yearly_return_summary
```

Out [245...

Ticker	AAPL	ADBE	AMD	LMT	MSFT	NVDA	
2000	-0.734227	0.776116	-0.108871	0.707128	-0.627882	0.399862	0.0
2001	0.472267	-0.334549	0.103304	0.383753	0.527377	3.469297	0.0
2002	-0.384978	-0.219961	-0.605857	0.258458	-0.228819	-0.828974	0.0
2003	0.443917	0.528984	1.125535	-0.115966	0.027999	0.884650	0.0
2004	2.026314	0.606887	0.481830	0.115084	0.088101	0.020797	0.0
2005	1.271764	0.198484	0.429239	0.193944	-0.010124	0.550468	0.0
2006	0.134983	0.067497	-0.371914	0.471730	0.128613	0.936682	0.0
2007	1.363723	0.070391	-0.615779	0.161517	0.208429	0.414358	0.0
2008	-0.561948	-0.489571	-0.697479	-0.186290	-0.437856	-0.755529	0.0
2009	1.322094	0.597741	3.067227	-0.091734	0.534423	1.144661	0.0
2010	0.507219	-0.170127	-0.156701	-0.057144	-0.079441	-0.167117	0.0
2011	0.228874	-0.096516	-0.362456	0.207734	-0.047546	-0.123894	0.0
2012	0.305586	0.318866	-0.562044	0.178842	0.025976	-0.121147	0.0
2013	0.047508	0.561815	0.529644	0.663476	0.395438	0.286952	0.0
2014	0.426284	0.226176	-0.324051	0.361329	0.284229	0.286810	0.0
2015	-0.020822	0.298590	0.074906	0.157398	0.218785	0.664514	0.0
2016	0.123843	0.119387	3.093863	0.205591	0.165057	2.329197	0.0
2017	0.480425	0.693467	-0.100612	0.300130	0.397411	0.904264	0.0
2018	-0.070543	0.273157	0.681239	-0.156867	0.202191	-0.328455	-0.
2019	0.887425	0.468629	1.435475	0.508813	0.582592	0.734099	0.
2020	0.782400	0.495440	0.867821	-0.088323	0.399428	1.180241	-0.2
2021	0.380610	0.168377	0.559046	0.062464	0.557899	1.244762	0.0
2022	-0.281995	-0.403707	-0.568890	0.408920	-0.276872	-0.514370	2.8
2023	0.547982	0.770747	1.302562	-0.025131	0.583498	2.460983	0.4
2024	0.355563	-0.233403	-0.128374	0.093269	0.145016	1.788741	-0.2
2025	0.119908	-0.206372	0.775346	0.032509	0.163905	0.348789	0.0
2026	0.286582	-0.208461	1.312570	0.237495	0.116077	0.119691	1.2

Task 3

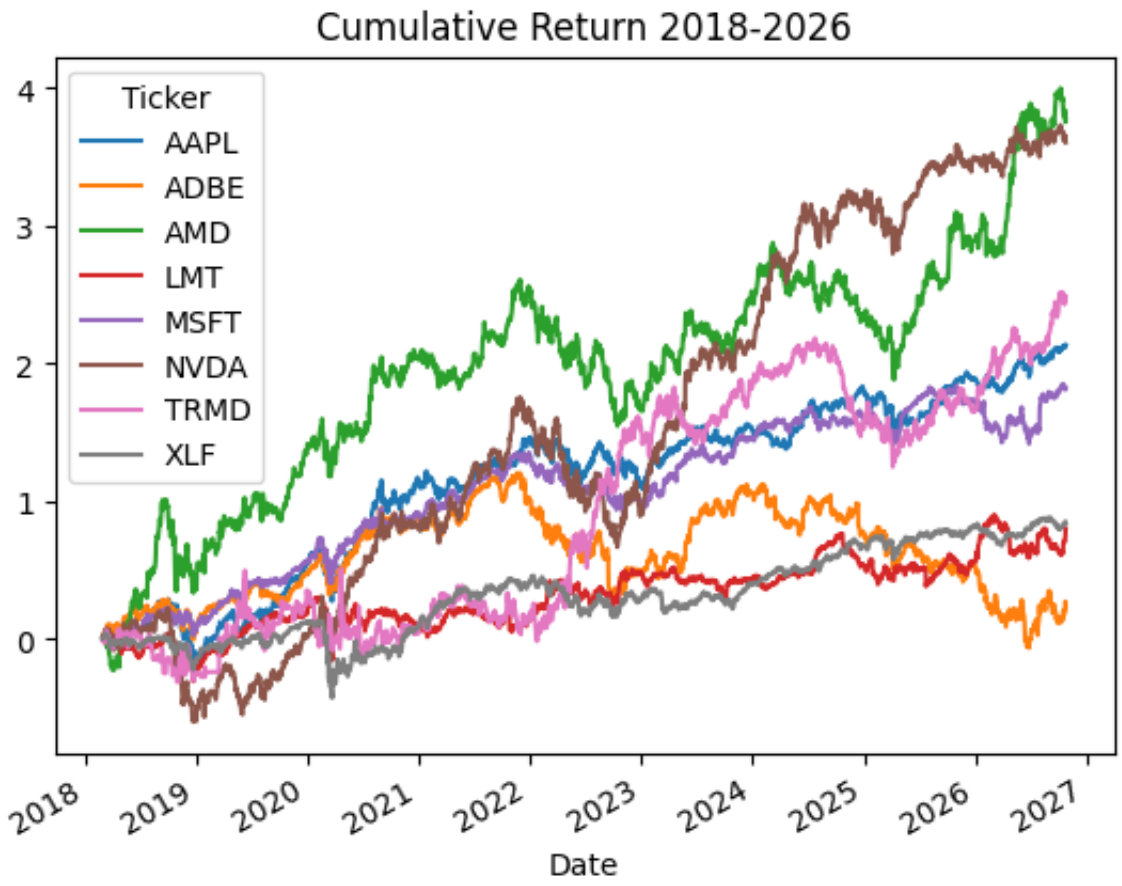
Back to Top

Data Visualization

Data Visualization

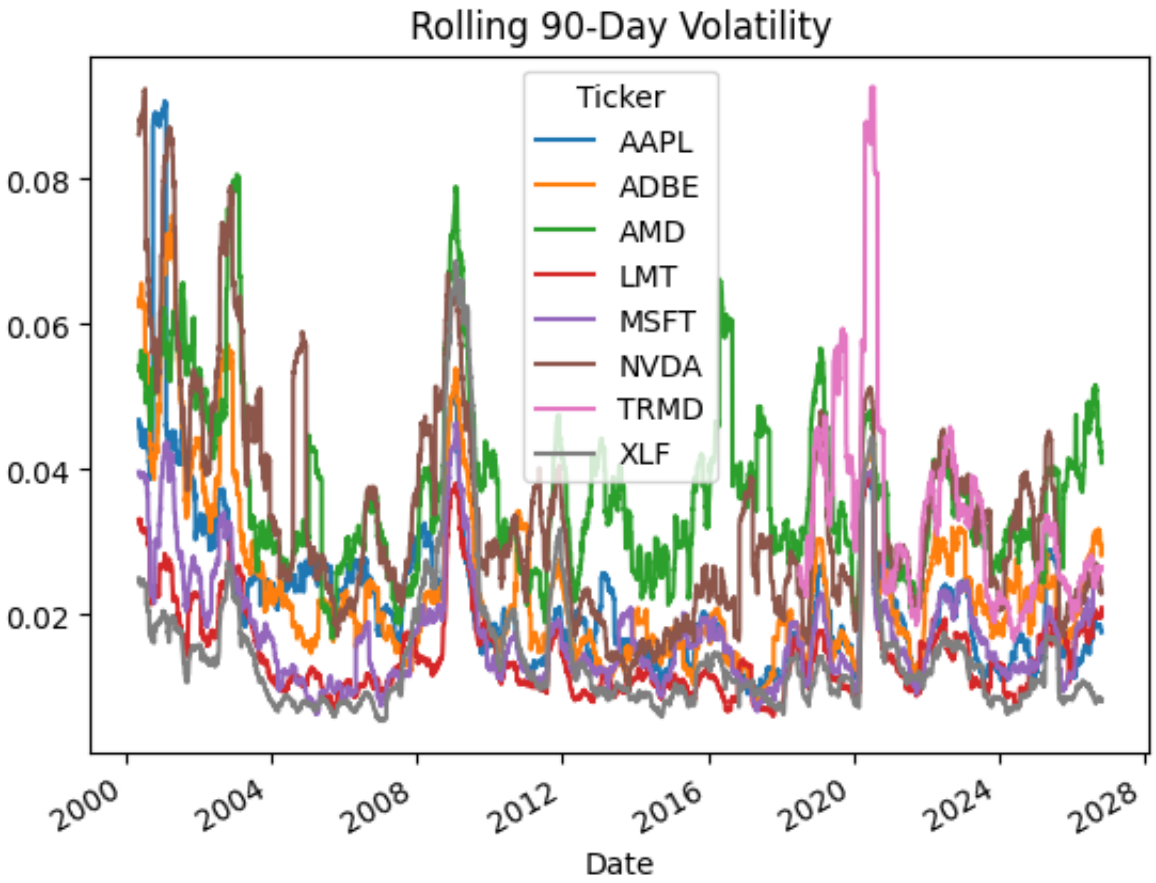
In [246…
```
log_returns[log_returns.index > '03-01-2018'].cumsum().plot(title='Cum
```

Out[246…
```
<Axes: title={'center': 'Cumulative Return 2018-2026'}, xlabel='Date'
>
```



In [248…
```
log_returns.rolling(window=90).std().plot(title='Rolling 90-Day Volati
```

Out[248…
```
<Axes: title={'center': 'Rolling 90-Day Volatility'}, xlabel='Date'>
```



Correlation Matrix

In [249…

```
log_returns.corr()
```

Out [249…

Ticker	AAPL	ADBE	AMD	LMT	MSFT	NVDA	TRMD
Ticker							
AAPL	1.000000	0.418488	0.356143	0.179366	0.456748	0.407097	0.099042
ADBE	0.418488	1.000000	0.365070	0.214215	0.538793	0.441653	0.049759
AMD	0.356143	0.365070	1.000000	0.150427	0.397393	0.497858	0.083286
LMT	0.179366	0.214215	0.150427	1.000000	0.238703	0.136005	0.066637
MSFT	0.456748	0.538793	0.397393	0.238703	1.000000	0.479276	0.068947
NVDA	0.407097	0.441653	0.497858	0.136005	0.479276	1.000000	0.091890
TRMD	0.099042	0.049759	0.083286	0.066637	0.068947	0.091890	1.000000
XLF	0.388956	0.435302	0.369273	0.392232	0.489246	0.362183	0.121257

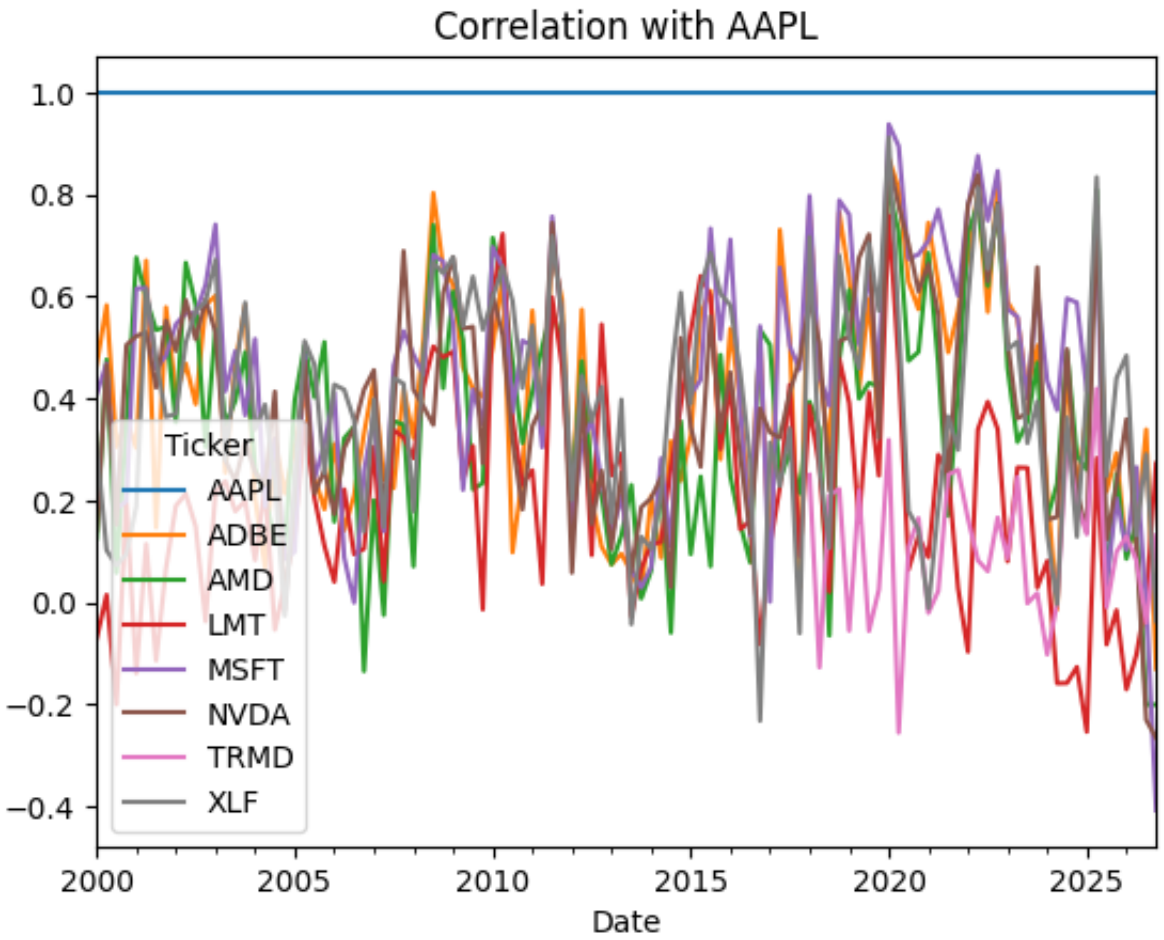
In [250…

```
yearly_corr = log_returns.groupby(log_returns.index.to_period("Q")).co

yearly_corr.xs('AAPL', level=1).plot(title='Correlation with AAPL')
```

Out [250…

```
<Axes: title={'center': 'Correlation with AAPL'}, xlabel='Date'>
```



Task 4

Back To Top

Written Assesment

Presented here is a dynamic and interactive data analysis document. The user can build a portfolio of tickers to perfrom analysis and run the coding cells producing analysis. It all works sequentially, so one can perform another analysis by just going back to the top and choosing a different date range and set of tickers.

Comparing 2-3 stocks, intially I chose Apple, Lockheed Martin, and TORM (A shipping tanker company). These different tickers exhibit different return and volatility characteristics. You can see that Lockheed Martin exhibits the lowest volatility, as a large stable company in the defense sector, this makes sense. Compared to Apple in the technology sector, we can see Apple's higher-cumulative return matches their higher volatility. If you scan the correlation matrix you can see companies like Apple, Microsoft, Nividia exhibit high correlation overall on a large time-period, which makes sense as they are all technology companies. All these companies exhibit a lower correlation with Lockheed Martin, an aerospace and defense industrial conglomerate. This makes sense, as this sector is more defensive, in the financial sense of the word 'defensive', then technology growth stocks.

What is interesting is quickly in the last few years other blue-chip technology companies have started to show negative correlation with Apple. This seems to be an second-order effect of how the competition surrounding Artificial Intelligence is impacting tech-sector stocks. This sort of observation would warrant a further analysis.

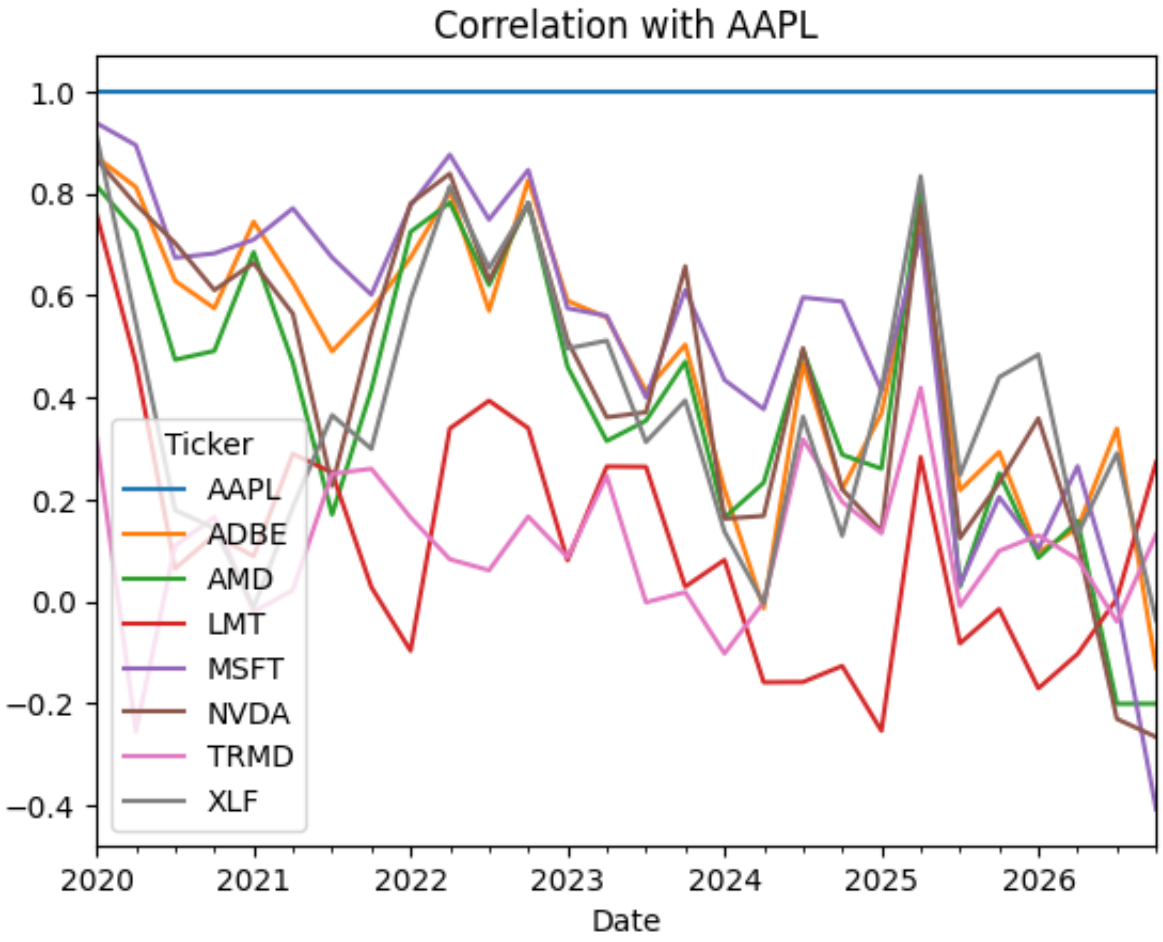
From a macroeconomic perspective you can see the impact of large exogenous events on all of these stocks. Most notably 2000 era Dot-Com crash in Apple. Volatility spikes and negative returns in 2007,2008 due to the Great Financial Crisis. You can also note 2020 COVID-era crash and subsequent rally, followed by 2022 interest-rate hiking adding volatility. More recently you can see 2025 "Liberation Day" volatility spikes from President Trump's aggressive tarrif policy announcements. As an tanker company that deals in oil-markets, you can see Torm's stock rally during 2022 Russian oil embargo, and sinking of oil vessels in Red Sea and 2026 war on Iran. TRMD's exposure to these regional crises has been limited, while their company has benefited from the higher-price point

demanded in international markets for shipping tankers.

In [251…
```
corr_analysis = log_returns[log_returns.index > '2020']
corr_analysis = corr_analysis.groupby(corr_analysis.index.to_period("Q

corr_analysis.xs('AAPL', level=1).plot(title='Correlation with AAPL')
```

Out[251…
```
<Axes: title={'center': 'Correlation with AAPL'}, xlabel='Date'>
```



In [ ]: